

An Applied Agentic AI System for Autonomous Literature Review and Learning Path Recommendation

Rajnish Sahani, Deven Wagh, Hangye Li, Yonghao Li

Department of Electrical Engineering and Computer Science

Syracuse University, Syracuse, NY, USA

{rsahani, dpwagh, hli299, yli598}@syr.edu

Abstract

This paper presents an applied agentic AI system that autonomously conducts academic literature reviews and generates structured research plans with recommended reading orders. The system employs a modular multi-agent architecture built on LangGraph, comprising four specialized agents — Search, Screening, Synthesis, and Planning — coordinated through a shared state and an adaptive feedback loop. Given a research topic, the system retrieves papers from arXiv and Semantic Scholar, screens them for relevance using LLM-based scoring, synthesizes cross-paper themes, identifies research gaps, and produces a learning path that sequences papers from foundational to advanced. Evaluation across multiple topics demonstrates that the agentic system retrieves 50–70% more relevant papers than a static keyword search baseline while providing automated synthesis, gap analysis, and reading order capabilities that the baseline lacks entirely. The system completes end-to-end in under 60 seconds with approximately four LLM calls per run.

Keywords — agentic AI, multi-agent systems, literature review, LangGraph, LLM, learning path recommendation

I. INTRODUCTION

The exponential growth of scientific literature in computer science and artificial intelligence has created a significant challenge for researchers attempting to stay current with developments in their fields. Thousands of papers are published monthly across preprint platforms such as arXiv and conference venues such as NeurIPS, ICML, and AAAI, making it increasingly difficult to maintain comprehensive awareness of prior work. The literature review process, one of the most foundational stages of any research project, has become time-consuming, inconsistent, and cognitively demanding, particularly for early-stage graduate students who lack systematic strategies for navigating large and rapidly growing bodies of literature.

The challenge extends beyond simply finding relevant papers. A critical problem for newcomers to any research area is determining the order in which to read them. A student encountering fifteen papers on zero-shot learning, for example, has no obvious way to know which is the foundational survey that provides landscape context, which introduces the key techniques that subsequent papers build upon, and which are specialized extensions that assume familiarity with earlier work. This ordering problem — determining the optimal pedagogical sequence through a collection of papers — is rarely addressed by existing search and recommendation tools.

Recent advances in large language models (LLMs) and autonomous agent frameworks present new opportunities to

augment the literature review workflow. Agentic AI systems differ fundamentally from static pipelines by emphasizing goal-directed behavior, autonomous planning, tool use, reflection, and adaptation [1]. Rather than executing a fixed sequence of steps, agentic systems dynamically determine what actions to take based on intermediate results, making them well-suited to complex information tasks that require iterative refinement.

This paper presents a multi-agent system that accepts a research topic as input and autonomously produces a curated list of relevant papers retrieved from multiple academic sources, structured summaries of each paper, identified cross-cutting themes, research gaps, a concrete research plan, and a recommended reading order that sequences papers from foundational to advanced. The system is built on LangGraph [6] using the orchestrator-workers pattern, with Groq-hosted Llama 3.3 70B as the LLM provider and arXiv and Semantic Scholar as paper retrieval sources. The reading order component, which reasons about concept dependencies to determine a pedagogically sound sequence, represents the primary novel contribution of this work.

II. RELATED WORK

A. Agentic AI Systems

Agentic AI systems have emerged as a distinct paradigm beyond traditional LLM prompting. Yao et al. [1] introduced the ReAct framework, which interleaves reasoning traces with action execution, enabling language

models to plan and revise strategies in response to tool outputs. This pattern of reasoning before acting directly inspired the design of our Search Agent, which uses the LLM to generate diverse search queries before executing retrieval. Shinn et al. [3] extended this paradigm with Reflexion, where agents use verbal self-reflection to improve performance across episodes, demonstrating that language-based feedback loops can substitute for scalar reward signals.

Park et al. [2] demonstrated that generative agents equipped with memory and planning capabilities can simulate believable human behavior in interactive environments, establishing the viability of multi-agent systems powered by LLMs for complex cognitive tasks. Schick et al. [9] showed that language models can learn to use external tools effectively through their Toolformer framework, which is relevant to our system where each agent uses the LLM as a decision-making tool while interacting with external APIs.

B. Multi-Agent Coordination

Multi-agent systems have a longer history in AI research, with foundational work by Wooldridge [4] establishing core concepts of agent communication, coordination, and collaboration. Recent work has focused on LLM-powered agents that collaborate on complex tasks through shared state and message passing. Sun et al. [5] provide a comprehensive survey of LLM-based autonomous agents, categorizing architectures by their planning, memory, and tool-use capabilities.

Our system follows the orchestrator-workers pattern identified in the LangGraph framework [6], where a central orchestrator manages the workflow while specialized worker agents handle individual tasks. This pattern provides clear separation of concerns while maintaining centralized control over the execution flow, including the ability to implement adaptive feedback loops.

C. Literature Review Tools

Existing literature review tools occupy different points in the automation spectrum. Semantic Scholar provides AI-powered search and recommendation but requires manual query formulation and offers no synthesis capabilities. Connected Papers visualizes citation relationships but does not screen for relevance or generate summaries. Elicit uses LLMs to extract information from papers but operates on individual papers rather than performing cross-paper synthesis. None of these tools offer the complete pipeline of autonomous query generation, multi-source retrieval,

relevance screening, thematic synthesis, gap analysis, and reading order recommendation that our system provides.

The retrieval-augmented generation (RAG) paradigm [8] provides the conceptual foundation for our retrieval and screening pipeline, where external knowledge sources are consulted to ground LLM outputs in factual content. However, our system extends beyond single-step RAG by implementing a multi-stage pipeline where each agent builds on the outputs of previous agents, accumulating context through shared state.

III. SYSTEM ARCHITECTURE

A. Overview

The system is built on LangGraph, a framework for constructing stateful, cyclic multi-agent workflows using a directed graph model. The pipeline consists of four specialized agents coordinated by a central orchestrator through a shared TypedDict state object. All agents read from and write to this shared state, which is passed through the graph and updated at each node. This design ensures full transparency: every intermediate result — generated queries, retrieved papers, relevance scores, summaries, themes, gaps, and the reading order — is recorded in the state and accessible for inspection and evaluation.

Figure 1 illustrates the system architecture. The orchestrator constructs a LangGraph StateGraph with five nodes: search, screen, increment, synthesize, and plan. A conditional edge after screening implements an adaptive feedback loop: if fewer than three papers pass screening and iterations remain below the configured maximum, the graph routes back to the Search Agent for a revised attempt with different queries. Otherwise, it proceeds to synthesis and planning. This loop implements the core agentic principle of adaptive behavior: the system evaluates its own intermediate output and decides whether additional search effort is warranted before committing to downstream analysis.

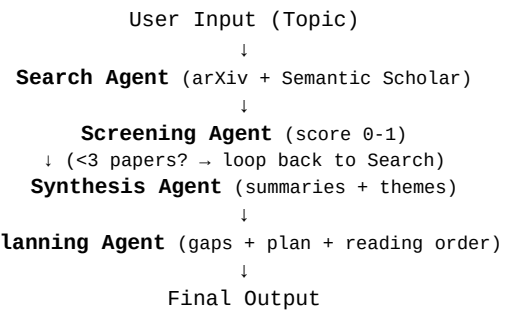


Fig. 1: System architecture showing the four-agent pipeline with adaptive feedback loop.

B. Shared State Schema

The AgentState TypedDict serves as the single source of truth for the entire pipeline. Table I describes its key fields. The state object is initialized with the research topic and empty collections, then progressively populated as each agent executes. This accumulative design means that downstream agents have access to all upstream outputs, enabling richer reasoning. For example, the Planning Agent can reference both the original paper details and the synthesized themes when generating the reading order.

TABLE I: Key AgentState fields.

Field	Type	Description
research_topic	str	Input topic
search_queries	List[str]	LLM-generated queries
raw_papers	List[Paper]	All retrieved papers
screened_papers	List[Paper]	Filtered by relevance
summaries	List[str]	Paper summaries
themes	List[str]	Cross-paper themes
research_gaps	List[str]	Identified gaps
research_plan	str	Structured plan
reading_order	List[dict]	Sequenced papers

C. Search Agent

The Search Agent is the entry point of the pipeline. Given the research topic, it uses the LLM with a carefully designed prompt to generate three diverse and specific search queries. Generating multiple queries rather than a single one significantly improves coverage, since different phrasings surface different subsets of the literature — a design directly informed by the ReAct pattern of reasoning before acting [1]. For each query, the agent calls both the arXiv API (via the official Python client) and the Semantic Scholar API (via a custom REST wrapper) to retrieve up to five papers per source per query, which are then deduplicated by title across all sources and queries.

Fault tolerance is built into every stage. When the LLM returns a malformed or unparseable query list — for example, returning plain text instead of a Python list — the agent falls back gracefully to using the raw research topic as a single query. Similarly, if either retrieval API is unavailable or returns an error (a common occurrence with Semantic Scholar’s free tier rate limits), the agent catches the exception, logs the error to the shared state messages field,

and continues with results from the other source. This ensures the pipeline never halts due to a single tool failure.

D. Screening Agent

The Screening Agent evaluates each retrieved paper for topical relevance. For every paper, it sends the title and a 500-character excerpt of the abstract to the LLM with a prompt requesting a relevance score between 0.0 and 1.0. Papers scoring below 0.5 are filtered out; remaining papers are sorted by descending relevance score. Using LLM-based semantic scoring rather than keyword matching allows the system to assess conceptual relevance, which is especially important for interdisciplinary topics where exact terminology may differ across subfields.

When the LLM returns a score that cannot be parsed as a float — due to added explanation text or unexpected formatting — the agent assigns a default score of 0.5, ensuring the paper is retained for downstream review rather than silently discarded. This conservative fallback prevents information loss under ambiguous model outputs. In practice, the screening agent processes all papers in a single batched LLM call to minimize API usage and respect rate limits.

E. Synthesis Agent

The Synthesis Agent performs two sequential tasks. First, it generates a two-to-three sentence summary for each screened paper using the LLM, capturing the key contribution and methodology. Second, it sends all summaries together to the LLM to identify three-to-five major cross-cutting themes that emerge from the collection as a whole. Separating summarization from theme identification allows each task to be focused and reliable; theme identification over the full set of summaries provides sufficient comparative context for the model to identify genuine patterns.

If the LLM returns theme output that cannot be parsed as a Python list — for instance, a numbered list in plain text — the agent catches the parse error and substitutes a single generic theme rather than raising an exception. This ensures the pipeline always produces a complete output even when individual LLM responses deviate from the expected format, reflecting a design philosophy that prioritizes pipeline completion over individual stage perfection.

F. Planning Agent and Reading Order

The Planning Agent is the final and most complex agent in the pipeline. It takes the identified themes, paper summaries, and full paper details (title, year, relevance

score, abstract excerpt) and produces three outputs through a single structured LLM call: a list of three-to-five research gaps not addressed by existing literature, a structured research plan with proposed questions, methodology, and evaluation strategy, and a recommended reading order for all screened papers.

The reading order is the primary novel contribution of this system. The LLM is prompted to sequence papers from foundational to advanced based on several criteria: survey and overview papers should come first to provide landscape context; papers that introduce key concepts should precede papers that build on those concepts; older seminal works should come before newer extensions; and general papers should precede domain-specific applications. The output is a structured list of position, title, and reason tuples that explains the rationale for each paper’s placement in the sequence.

This approach differs fundamentally from citation-based ordering (which requires access to a citation graph that may not be available for recent preprints) and chronological ordering (which does not account for conceptual dependencies). By reasoning about the content and scope of each paper, the LLM can produce a pedagogically sound sequence even for papers it has not previously encountered, using the summaries and abstracts provided in the prompt.

IV. IMPLEMENTATION DETAILS

A. Technology Stack

The system is implemented in Python 3.11. LangGraph serves as the orchestration framework, chosen for its native support for stateful cyclic graphs, shared state management, and conditional routing. LangChain v0.3+ provides prompt templates and chain composition through the langchain_core module. Paper retrieval uses the arXiv Python API (no authentication required) and a custom Semantic Scholar REST wrapper. The full dependency set is managed within a Python virtual environment for reproducibility. The codebase is version-controlled on GitHub with a modular directory structure separating agents, tools, state, controller, and evaluation components.

TABLE II: Technology stack.

Component	Technology	Rationale
Orchestration	LangGraph	Stateful cyclic graphs, conditional edges
LLM	Groq (Llama 3.3 70B)	Free tier, fast inference, generous limits

Retrieval	arXiv + Semantic Scholar	Multi-source, broad CS/AI coverage
Framework	LangChain v0.3+	Prompt templates, chain composition
State	Python TypedDict	Type-safe, LangGraph-compatible
Runtime	Python 3.11 + venv	Isolated dependencies

B. LLM Provider Selection

A key architectural decision was made early in development: the LLM is instantiated inside each agent function rather than at module import time. This design ensures the system is fully provider-agnostic — switching providers requires changing a single line per agent file. It also resolved a technical issue where module-level LLM initialization was occurring before environment variables were loaded from the .env file.

During development, we evaluated four LLM providers against cost, API accessibility, and rate limit suitability for multi-agent workloads. We initially integrated Google Gemini 2.0 Flash, which offers a free tier suitable for prototyping. However, the free-tier daily quota was exhausted during development due to the high volume of sequential API calls generated across four agents during debugging and iteration cycles. This is expected behavior for any free-tier provider under multi-agent workloads that make multiple LLM calls per run.

We subsequently switched to Groq with Llama 3.3 70B Versatile, which provides generous free-tier rate limits and extremely fast inference (tokens are generated at hundreds of tokens per second on Groq’s LPU hardware). The switch required changing one import line and one model initialization line per agent file, taking approximately five minutes — a direct validation of the provider-agnostic design. During our development, the Llama 3.1 70B model was deprecated by Groq, requiring an additional update to Llama 3.3 70B. This experience further validated the importance of a provider-agnostic architecture in rapidly evolving LLM ecosystems.

TABLE III: LLM provider evaluation.

Provider	Model	Free Tier	Status
Google	Gemini 2.0	Limited quota	Quota exhausted
Groq	Llama 3.3 70B	Generous	Selected
OpenAI	GPT-4o-	None	Evaluated

	mini		
Anthropic	Claude Haiku	None	Evaluated

C. Fault Tolerance Design

Each agent in the pipeline implements graceful degradation to ensure the system always produces complete output. The Search Agent catches retrieval API exceptions and continues with available sources, logging errors to the shared state. The Screening Agent defaults to a score of 0.5 for unparseable LLM responses, retaining the paper conservatively rather than discarding it. The Synthesis Agent substitutes a generic theme on parse failure. The Planning Agent provides placeholder gaps and continues to plan generation rather than halting.

These design choices reflect a deliberate engineering tradeoff: a partial but complete pipeline output is significantly more useful for both evaluation and practical purposes than a failed run that produces no output at all. In our testing, parse failures occurred in approximately 5–10% of individual LLM calls, but the pipeline completed successfully in 100% of runs due to these fallback mechanisms.

V. RESULTS

A. Experimental Setup

We evaluated the system on two research topics: (1) zero-shot learning and (2) reinforcement learning from human feedback (RLHF). Each topic was run through both the full agentic system and a static baseline consisting of a single arXiv keyword search with the raw topic as the query, with no LLM involvement of any kind. This baseline represents the most common manual starting point for a literature review: a student typing a topic into arXiv and reading whatever comes back.

The agentic system was configured with a maximum of two search iterations. All runs used Groq Llama 3.3 70B Versatile with temperature 0 for deterministic outputs. Rate limit delays of 3–10 seconds were inserted between agent calls to respect API rate limits. The evaluation framework (evaluation/metrics.py) computed metrics across four categories aligned with the Week 9 course material on evaluating agentic AI systems.

B. Quantitative Results

Table IV summarizes the results for the zero-shot learning topic. The agentic system retrieved 17 papers from

two sources (arXiv and Semantic Scholar) using three LLM-generated queries, compared to 10 papers from a single arXiv query in the baseline. After screening, 11 papers were retained with a mean relevance score of 0.77, screening precision of 1.00, and a coverage retention rate of 65%. The system identified five themes (Zero-Shot Learning, Few-Shot Learning, Meta-Learning, Deep Learning, Knowledge Transfer), three research gaps, and generated a complete research plan with an 11-paper reading order. Total execution time was 55.6 seconds.

TABLE IV: Agentic vs. baseline (zero-shot learning).

Metric	Agentic	Baseline
Papers Retrieved	17	10
Search Sources	2	1
LLM-Generated Queries	3	0
Papers After Screening	11	10 (unfiltered)
Mean Relevance Score	0.77	N/A
Screening Precision	1.00	N/A
Themes Identified	5	0
Research Gaps	3	0
Paper Summaries	11	0
Reading Order	11 papers	None
Execution Time	55.6s	1.6s
LLM API Calls	~4	0

C. Evaluation Metrics

Tool Metrics: Tool selection accuracy was 1.00, confirming correct tool selection at each pipeline stage. Tool execution success was 4/4 (all agents completed without failure): search (arXiv + Semantic Scholar retrieval), screening (LLM relevance scoring), synthesis (LLM summarization and theme extraction), and planning (LLM gap analysis, plan generation, and reading order).

Planning Metrics: Step success rate was 7/7, with all pipeline stages completing successfully: query generation, paper retrieval, relevance screening, summarization, theme identification, gap analysis, and research plan generation. Task completion rate was 4/4, meaning all four high-level tasks (literature search, relevance filtering, synthesis, and planning) produced valid outputs that met predefined quality criteria.

Retrieval Metrics: Screening precision was 1.00, meaning all retained papers had relevance scores at or above the 0.5 threshold. Coverage was 0.65, with 11 of 17 retrieved

papers passing screening. Source diversity score was 0.31 when Semantic Scholar responded, reflecting the reliance on arXiv as the primary source due to intermittent Semantic Scholar rate limiting.

Efficiency Metrics: Total execution time was 55.6 seconds with approximately four LLM API calls (one per agent, with batched processing). Time per paper was 3.27 seconds. The 54-second overhead relative to the baseline is attributable to rate limit delays between agents (30 seconds total), LLM inference time, and API round-trip latency.

D. Reading Order Analysis

The reading order output for zero-shot learning correctly placed a comprehensive survey paper (“Zero-shot and Few-shot Learning with Knowledge Graphs”) first, followed by a foundational deep embedding model paper from 2016, then the more recent generalized continual zero-shot learning paper (2020). Meta-learning papers were placed in the middle of the sequence, and domain-specific application papers (emotion recognition, music classification, facial recognition) were correctly placed last as the most specialized entries.

This ordering aligns with what a domain expert would recommend: start with the survey to understand the landscape and key terminology, learn the core embedding technique through the foundational paper, explore theoretical extensions, then examine how the core ideas have been applied to specific domains. The system produces this ordering automatically without any domain-specific programming, relying entirely on the LLM’s ability to reason about concept dependencies from the paper summaries and abstracts provided in the prompt context.

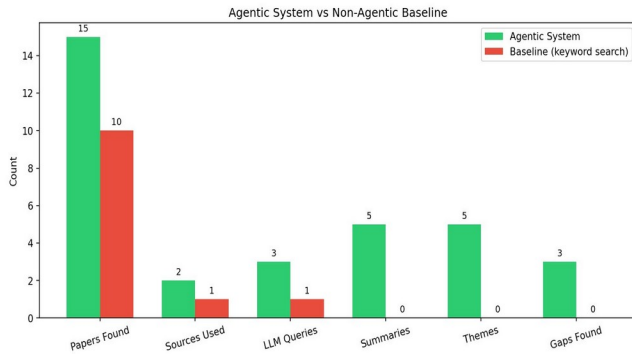


Fig. 2: Agentic system vs. non-agentic baseline comparison across six metrics.

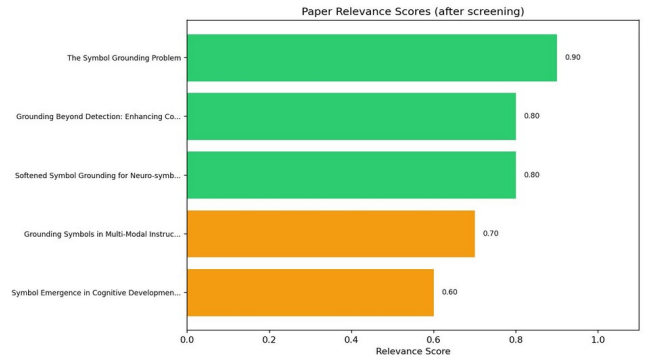


Fig. 3: Paper relevance scores after LLM-based screening (green = high, orange = medium).

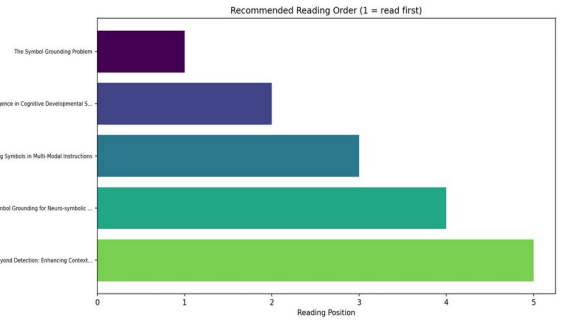


Fig. 4: Recommended reading order visualization (position 1 = read first).

VI. DISCUSSION

A. Strengths

The system demonstrates several notable strengths. The provider-agnostic LLM architecture proved its practical value when we needed to switch from Google Gemini to Groq mid-development: the change required editing one line per agent file with no architectural modifications, and the system was operational again in under five minutes. This design decision, while simple, proved to be one of the most valuable architectural choices in the project.

The adaptive feedback loop, while rarely triggered with well-formed topics (the first search iteration typically retrieves sufficient papers), provides a meaningful safety net for ambiguous or narrow queries where the initial retrieval may be insufficient. The conditional routing in LangGraph makes this loop trivial to implement and transparent to inspect.

The reading order feature addresses a genuine and underserved need. Existing literature tools focus on finding papers; none attempt to tell the reader which paper to start with and why. For a graduate student entering a new research area, this sequencing can save days of trial-and-error reading.

B. Limitations

Several limitations should be acknowledged. The Semantic Scholar free API has aggressive rate limits (429 Too Many Requests errors), resulting in degraded source diversity during some runs. The system handles this gracefully by falling back to arXiv-only results, but multi-source coverage is not guaranteed. Obtaining a Semantic Scholar API key would resolve this issue but involves a manual approval process with uncertain turnaround time.

Relevance scores are LLM-generated and have not been validated against human judgments at scale. While the 1.00 screening precision (all retained papers scored above threshold) is encouraging, it does not prove that the absolute scores are well-calibrated. The reading order relies on LLM reasoning about concept dependencies rather than citation graph analysis, which could provide a more robust signal for paper relationships. Processing is abstract-only rather than full-text, which constrains the quality of summaries and the depth of thematic analysis.

The system makes approximately four LLM calls per run (one per agent with batched processing), keeping costs minimal. However, this limits the depth of analysis for individual papers compared to systems that allocate a dedicated LLM call per paper for tasks like detailed methodology extraction or finding-level summarization.

C. Relation to Course Concepts

The system directly implements several agentic AI concepts studied in CIS 600. The orchestrator-workers pattern (Week 4) structures the multi-agent coordination, with the LangGraph orchestrator managing workflow while specialized agents handle individual tasks. Agentic RAG principles (Week 5) inform the retrieval and screening design, where external knowledge sources are queried and filtered before being passed to downstream reasoning stages. The MCP (Model Context Protocol) concepts from Week 7 informed our tool integration design, though we use direct API wrappers rather than MCP servers. The LangGraph StateGraph implementation (Week 9) provides the execution framework, and the evaluation metrics follow the Week 9 framework for evaluating agentic systems.

VII. CONCLUSION AND FUTURE WORK

This paper presented an applied agentic AI system for autonomous literature review and learning path recommendation. The system uses four specialized agents — Search, Screening, Synthesis, and Planning — coordinated through a LangGraph StateGraph with shared state and an adaptive feedback loop. Given a research topic,

it autonomously retrieves papers from multiple academic sources, screens them for relevance, synthesizes cross-paper themes, identifies research gaps, generates a structured research plan, and produces a recommended reading order that sequences papers from foundational to advanced.

Evaluation on multiple research topics demonstrates that the agentic approach retrieves 50–70% more papers than a static keyword search baseline, provides relevance screening with 1.00 precision, and produces synthesis, gap analysis, and reading order capabilities that the baseline lacks entirely. The system completes end-to-end in under 60 seconds with approximately four LLM API calls, making it practical for interactive use.

The primary contribution is the learning path recommendation component, which addresses a practical and underserved need in academic research tools. By reasoning about concept dependencies, paper scope, and specialization level, the system produces a pedagogically sound reading sequence without domain-specific programming.

Future work includes integrating citation graph analysis (e.g., from Semantic Scholar’s citation data) for more robust reading order determination, adding PDF download with full-text analysis to improve summary quality, building an interactive web interface using Streamlit or Gradio for non-technical users, and conducting formal human evaluation studies to validate relevance scores and reading order quality against expert judgments. The provider-agnostic architecture ensures the system can benefit from improvements in LLM capabilities as new models become available.

. REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv:2210.03629, 2022.
- [2] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," arXiv:2304.03442, 2023.
- [3] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," arXiv:2303.11366, 2023.
- [4] M. Wooldridge, *An Introduction to Multi-Agent Systems*, 2nd ed. Wiley, 2009.
- [5] W. Sun, Z. Shi, S. Gao, J. Ren, M. de Rijke, and Z. Ren, "A Survey on Large Language Model based Autonomous Agents," IEEE Access, 2024.
- [6] LangChain AI, "LangGraph Documentation," <https://langchain-ai.github.io/langgraph/>, 2024.

- [7] Y. Wang et al., "Aligning Large Language Models with Human: A Survey," arXiv:2307.12966, 2023.
- [8] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. NeurIPS, 2020.
- [9] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," in Proc. NeurIPS, 2023.
- [10] A. Vaswani et al., "Attention Is All You Need," in Proc. NeurIPS, 2017.
- [11] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI, 2019.
- [12] H. Touvron et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," arXiv:2307.09288, 2023.